



İhsan Doğramacı Bilkent University
Department of Computer Engineering

CS 353 - Database Systems
Spring 2026

Design Report

Team 2

Uğurcan Doğruçam	22302777
Eren Sucuoğlu	22302576
Talha Safa Küçük	22302802
Ahmet Güneyli	22302556
Hidayet Başer	22302003

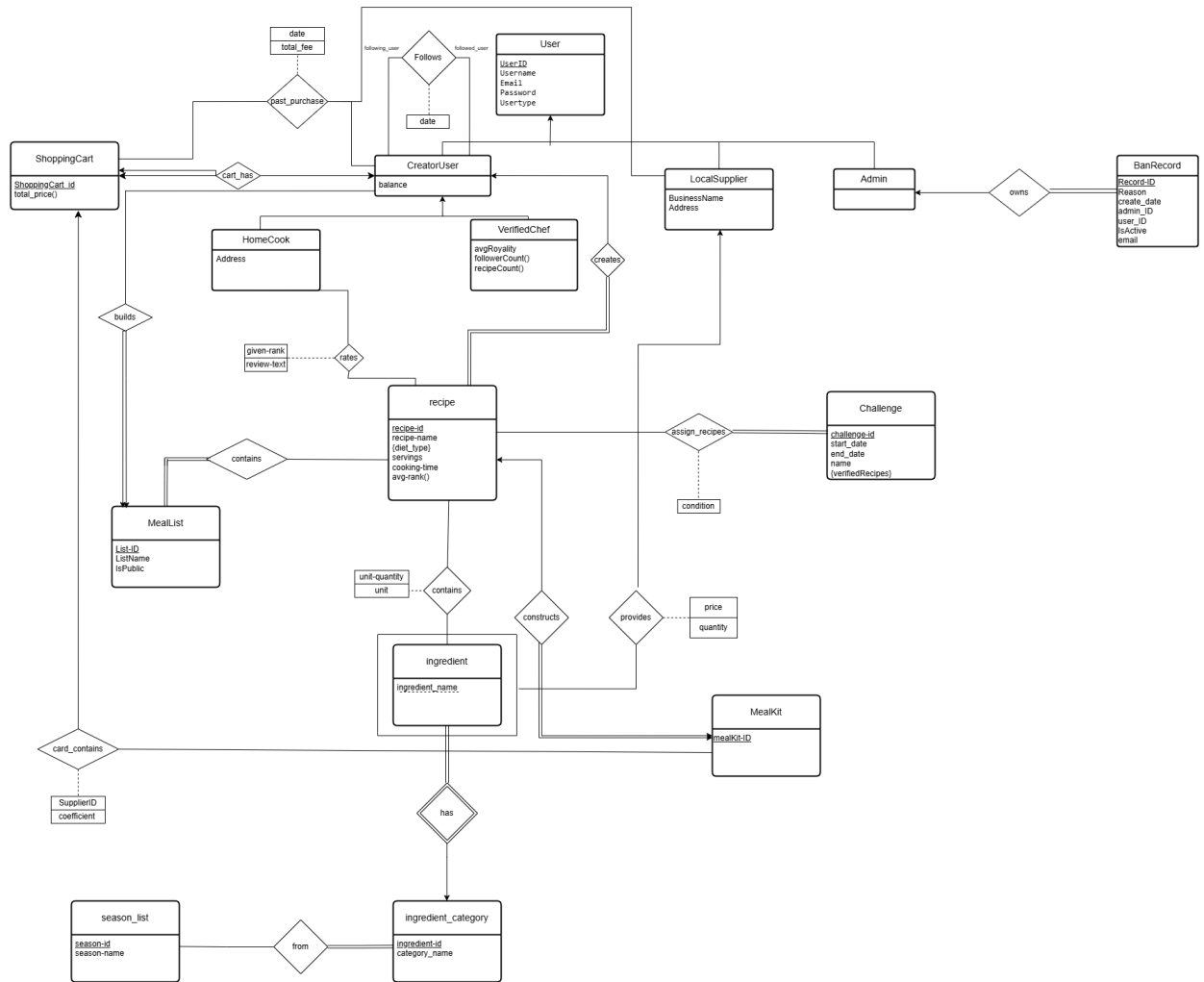
Project Topic: Culinary Discovery & Meal Kit Marketplace

Date: March 24, 2026

Contents

1	Revised E/R Diagram	2
2	Table Schemas (Relational Model)	3
2.1	User Related Tables	3
2.2	User Relationship Tables	3
2.3	Shopping and MealList Tables	3
2.4	Recipe and Inventory Tables	4
2.5	MealKit and Supply Tables	4
2.6	Challenge Tables	4
2.7	Season and Category Tables	5
3	User Interface Design and SQL Queries	5
3.1	Login and Register Functionalities	5
3.1.1	SQL Query: User Login	5
3.1.2	SQL Query: Home Cook Registration	6
3.1.3	SQL Query: Verified Chef Registration	6
3.1.4	SQL Query: Supplier Registration	6
3.2	Additional Functionality	7
3.2.1	SQL Query: SEASON LIST	8
3.2.2	SQL Query: FOLLOWS	9
3.3	Recipe Discovery, Scaling, and "Shop This Meal"	10
3.3.1	User Interface Flow From Recipe Discovery to Order Confirmation	10
3.3.2	SQL Query: Apply filters based on dietary restrictions, cooking time, ingredients, and chef rating; and list matching recipes	14
3.3.3	SQL Query: The Home Cook selects a recipe to view details	14
3.3.4	SQL Query: User clicks "Shop This Meal"	15
3.3.5	SQL Query: Checkout and Payment	17
4	Advanced Database Components	18
4.1	Views	18
4.1.1	Chef Royalty Dashboard View	18
4.1.2	Active Cart Summary View	19
4.2	Triggers	20
4.2.1	Automotic Royalty Update	20
4.3	Constraints	21
4.3.1	Recipe Rating Range	21
5	Implementation Plan	21
5.1	Next.js with TypeScript for Backend	21
5.2	Next.js and Tailwind CSS for Frontend	21
5.3	PostgreSQL for Database Management	21
5.4	Authentication with NextAuth.js	22
5.5	Integration and Raw SQL Connections	22

1 Revised E/R Diagram



2 Table Schemas (Relational Model)

The conceptual E/R design is mapped to the following relational schema. Primary keys are underlined and foreign keys are denoted as (FK). Derived attributes (computed, not stored) are marked with ().

2.1 User Related Tables

- **User**(userID, username, email, password, userType)
- **CreatorUser**(userID, balance)
userID (FK) references User.userID
- **HomeCook**(userID, address)
userID (FK) references CreatorUser.userID
- **VerifiedChef**(userID, avgRoyalty)
userID (FK) references CreatorUser.userID
Derived: followerCount(), recipeCount()
- **LocalSupplier**(userID, businessName, address)
userID (FK) references User.userID
- **Admin**(userID)
userID (FK) references User.userID

2.2 User Relationship Tables

- **Follows**(followingUserID, followedUserID, date)
followingUserID (FK) references User.userID
followedUserID (FK) references User.userID
CONSTRAINT: followingUserID ≠ followedUserID
- **BanRecord**(recordID, reason, createDate, adminID, userID, isActive, email)
adminID (FK) references Admin.userID
userID (FK) references User.userID

2.3 Shopping and MealList Tables

- **ShoppingCart**(shoppingCartID, homecookID)
homecookID (FK) references HomeCook.userID
Derived: total_price()
- **PastPurchase**(homecookID, shoppingCartID, date, totalFee)
homecookID (FK) references HomeCook.userID
shoppingCartID (FK) references ShoppingCart.shoppingCartID
- **MealList**(listID, listName, isPublic, homecookID)
homecookID (FK) references HomeCook.userID
- **MealListContains**(listID, recipeID)
listID (FK) references MealList.listID
recipeID (FK) references Recipe.recipeID

2.4 Recipe and Inventory Tables

- **Recipe**(recipeID, recipeName, servings, cookingTime, chefID)
chefID (FK) references VerifiedChef.userID
Derived: avg_rank()
- **RecipeDietType**(recipeID, dietType)
recipeID (FK) references Recipe.recipeID
(Multivalued attribute {diet_type} of Recipe)
- **Rates**(homecookID, recipeID, givenRank, reviewText)
homecookID (FK) references HomeCook.userID
recipeID (FK) references Recipe.recipeID
- **Ingredient**(ingredientName)
- **RecipeContainsIngredient**(recipeID, ingredientName, unitQuantity, unit)
recipeID (FK) references Recipe.recipeID
ingredientName (FK) references Ingredient.ingredientName

2.5 MealKit and Supply Tables

- **MealKit**(mealKitID)
- **Constructs**(recipeID, mealKitID)
recipeID (FK) references Recipe.recipeID
mealKitID (FK) references MealKit.mealKitID
- **Provides**(supplierID, mealKitID, price, quantity)
supplierID (FK) references LocalSupplier.userID
mealKitID (FK) references MealKit.mealKitID
- **CardContains**(shoppingCartID, mealKitID, supplierID, coefficient)
shoppingCartID (FK) references ShoppingCart.shoppingCartID
mealKitID (FK) references MealKit.mealKitID
supplierID (FK) references LocalSupplier.userID

2.6 Challenge Tables

- **Challenge**(challengeID, startDate, endDate, name)
- **ChallengeVerifiedRecipes**(challengeID, recipeID)
challengeID (FK) references Challenge.challengeID
recipeID (FK) references Recipe.recipeID
(Multivalued attribute {verifiedRecipes} of Challenge)
- **AssignRecipes**(recipeID, challengeID, condition)
recipeID (FK) references Recipe.recipeID
challengeID (FK) references Challenge.challengeID

2.7 Season and Category Tables

- **season_list**(seasonID, seasonName)
- **ingredient_category**(ingredientID, categoryName)
- **category_season**(ingredientID, seasonID)
ingredientID (FK) references ingredient_category.ingredientID
seasonID (FK) references season_list.seasonID

3 User Interface Design and SQL Queries

3.1 Login and Register Functionalities

The screenshot shows the 'Create an Account' page for FarmToTable. The page has a light beige background. At the top left is the FarmToTable logo, and at the top right are links for 'Discover', 'Login', and 'Register'. The main content is a white card with the title 'Create an Account' and a subtitle 'Join FarmToTable as a Home Cook, Chef, or Supplier'. The form includes fields for 'Full Name' (with example 'e.g. John Doe') and 'Email' (with example 'user@example.com'). There is a 'Password' field with a hint 'Create a password'. Below these is a 'Register As' section with three radio buttons: 'Home Cook' (selected), 'Chef', and 'Supplier'. There is a 'Delivery Address' field with example 'e.g. Silken, Ankara'. A large orange 'Register' button is at the bottom of the form. Below the button is a link 'G Register with Google'. At the very bottom, it says 'Already have an account? [Login here](#)'.

1) Register Page

The screenshot shows the 'Login to FarmToTable' page. The page has a light beige background. At the top left is the FarmToTable logo, and at the top right are links for 'Discover', 'Login', and 'Register'. The main content is a white card with the title 'Login to FarmToTable' and a subtitle 'Welcome back! Please enter your details.'. The form includes fields for 'Username or Email' (with hint 'Enter your email or username') and 'Password' (with hint 'Enter your password'). A large orange 'Login' button is at the bottom of the form. Below the button is a link 'G Login with Google'. At the very bottom, it says 'Don't have an account? [Register here](#)'.

2) Login Page

3.1.1 SQL Query: User Login

Following authentication via NextAuth.js, the following query identifies the specific user role and local profile details:

```
SELECT
    username,
    user_type
FROM "user"
WHERE user_id = $1;
```

3.1.2 SQL Query: Home Cook Registration

```
-- Step 1: Hash the password on the server side, then insert into
           User table
-- user_id is defined as SERIAL and is auto-generated by
           PostgreSQL
INSERT INTO "user" (name, email, password, user_type)
VALUES ('New User', 'user@example.com', $1, 'home_cook')
RETURNING user_id;

-- Step 2: Insert into specific HomeCook table using the returned
           user_id
INSERT INTO home_cook (user_id, address)
VALUES ($2, 'Bilkent, Ankara');
```

3.1.3 SQL Query: Verified Chef Registration

```
-- Step 1: Hash the password on the server side, then insert into
           User table
-- user_id is defined as SERIAL and is auto-generated by
           PostgreSQL
INSERT INTO "user" (name, email, password, user_type)
VALUES ('New User', 'user@example.com', $1, 'verified_chef')
RETURNING user_id;

-- Step 2: Insert into specific verified_chef table using the
           returned user_id
INSERT INTO verified_chef (user_id, avg_royalty)
VALUES ($2, 123);
```

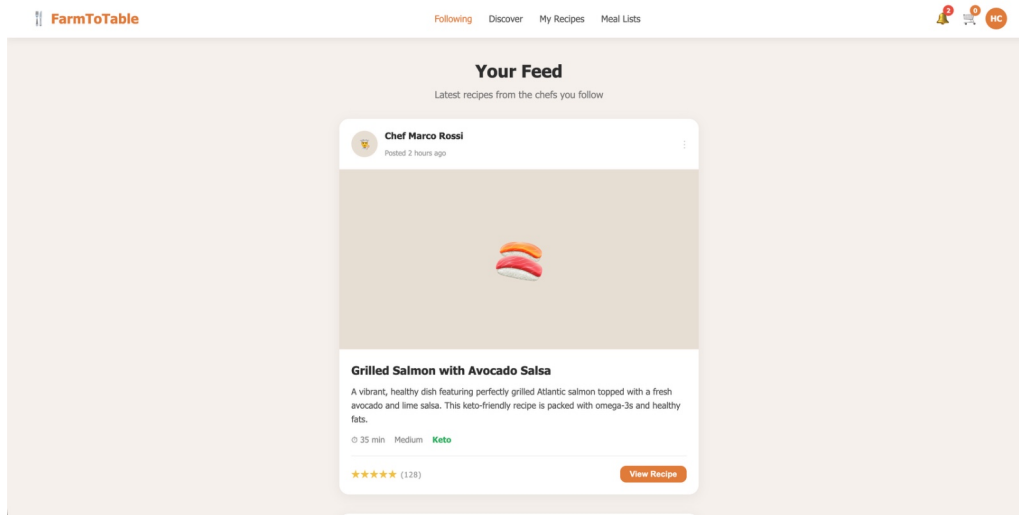
3.1.4 SQL Query: Supplier Registration

```
-- Step 1: Hash the password on the server side, then insert into
           User table
-- user_id is defined as SERIAL and is auto-generated by
           PostgreSQL
INSERT INTO "user" (name, email, password, user_type)
VALUES ('New User', 'user@example.com', $1, 'supplier')
RETURNING user_id;

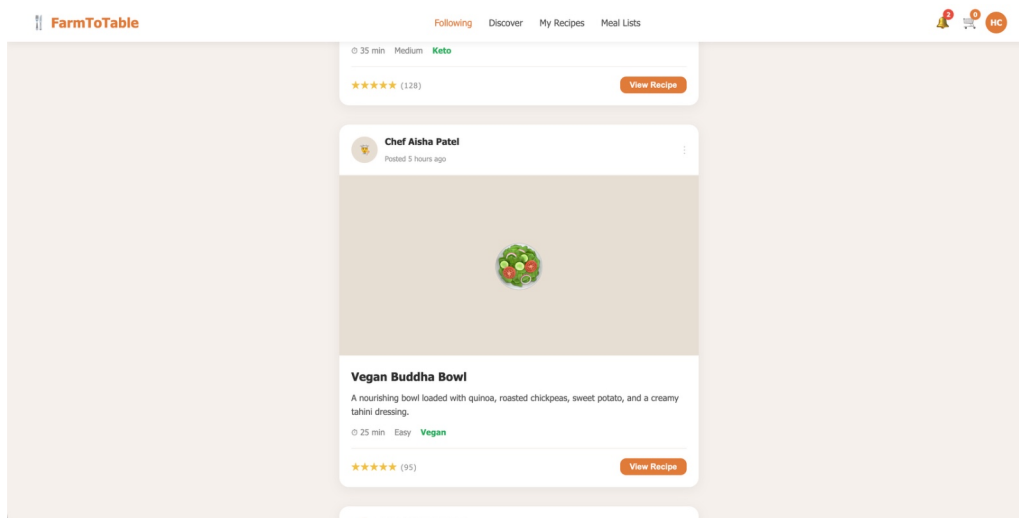
-- Step 2: Insert into specific supplier table using the returned
           user_id
```

```
INSERT INTO supplier (user_id, business_name, address)
VALUES ($2, 'Business Name', 'Bilkent, Ankara');
```

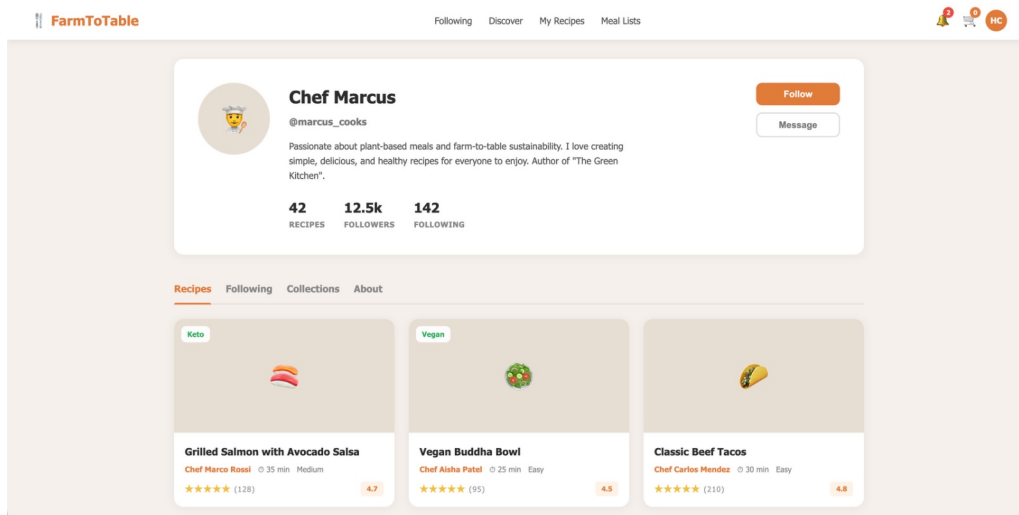
3.2 Additional Functionality



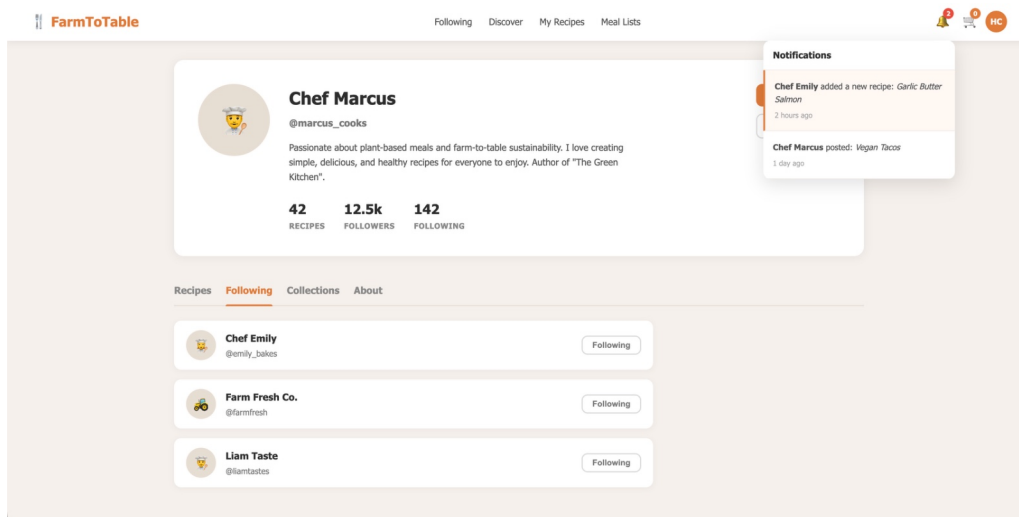
1) Follow feed page



2) Follow feed page



3) User Profile Page



4) Following Section Profile

3.2.1 SQL Query: SEASON LIST

```
-- INSERT: Add a new season
INSERT INTO season_list (season_id, season_name)
VALUES (1, 'Spring');

-- INSERT: Add a new ingredient category
INSERT INTO ingredient_category (ingredient_id, category_name)
VALUES (1, 'Leafy Greens');

-- INSERT: Link a category to a season
INSERT INTO category_season (ingredient_id, season_id)
VALUES (1, 1);

-- SELECT: Get all ingredient categories available in a given
season
SELECT ic.ingredient_id, ic.category_name, sl.season_name
FROM ingredient_category ic
```

```

JOIN category_season cs ON ic.ingredient_id = cs.ingredient_id
JOIN season_list sl ON cs.season_id = sl.season_id
WHERE sl.season_name = 'Spring';

-- SELECT: Get all seasons for a specific category
SELECT sl.season_name
FROM season_list sl
JOIN category_season cs ON sl.season_id = cs.season_id
JOIN ingredient_category ic ON cs.ingredient_id = ic.
    ingredient_id
WHERE ic.category_name = 'Leafy Greens';

-- UPDATE: Rename a season
UPDATE season_list
SET season_name = 'Early Spring'
WHERE season_id = 1;

-- DELETE: Remove a category-season association
DELETE FROM category_season
WHERE ingredient_id = 1 AND season_id = 1;

```

3.2.2 SQL Query: FOLLOWS

```

-- INSERT: HomeCook (ID=10) follows VerifiedChef (ID=5)
INSERT INTO follows (homecook_id, chef_id)
VALUES (10, 5);

-- SELECT: Get all chefs a specific homecook follows
SELECT u.Username, vc.avgRoyalty, f.follow_date
FROM follows f
JOIN User u ON f.chef_id = u.UserID
JOIN VerifiedChef vc ON f.chef_id = vc.UserID
WHERE f.homecook_id = 10
ORDER BY f.follow_date DESC;

-- SELECT: Get all followers of a specific chef
SELECT u.Username, f.follow_date
FROM follows f
JOIN User u ON f.homecook_id = u.UserID
WHERE f.chef_id = 5
ORDER BY f.follow_date DESC;

-- SELECT: Get follower count for each verified chef
SELECT u.Username, COUNT(f.homecook_id) AS follower_count
FROM VerifiedChef vc
JOIN User u ON vc.UserID = u.UserID
LEFT JOIN follows f ON vc.UserID = f.chef_id
GROUP BY u.Username
ORDER BY follower_count DESC;

```

```

-- SELECT: Check if a homecook already follows a chef
SELECT EXISTS (
    SELECT 1 FROM follows
    WHERE homecook_id = 10 AND chef_id = 5
) AS is_following;

-- DELETE: Unfollow a chef
DELETE FROM follows
WHERE homecook_id = 10 AND chef_id = 5;

```

3.3 Recipe Discovery, Scaling, and "Shop This Meal"

3.3.1 User Interface Flow From Recipe Discovery to Order Confirmation

FarmToTable Discover My Recipes Meal Lists Challenges

Discover Recipes
Find the perfect recipe by filtering based on your preferences

Dietary Restrictions
☒ Keto ☒ Vegan
☒ Vegetarian
☐ Gluten-Free
☐ Dairy-Free

Cooking Time (max minutes)
 10 min **60 min** 120 min

Ingredients (search)
 e.g. salmon, avocado, tofu...

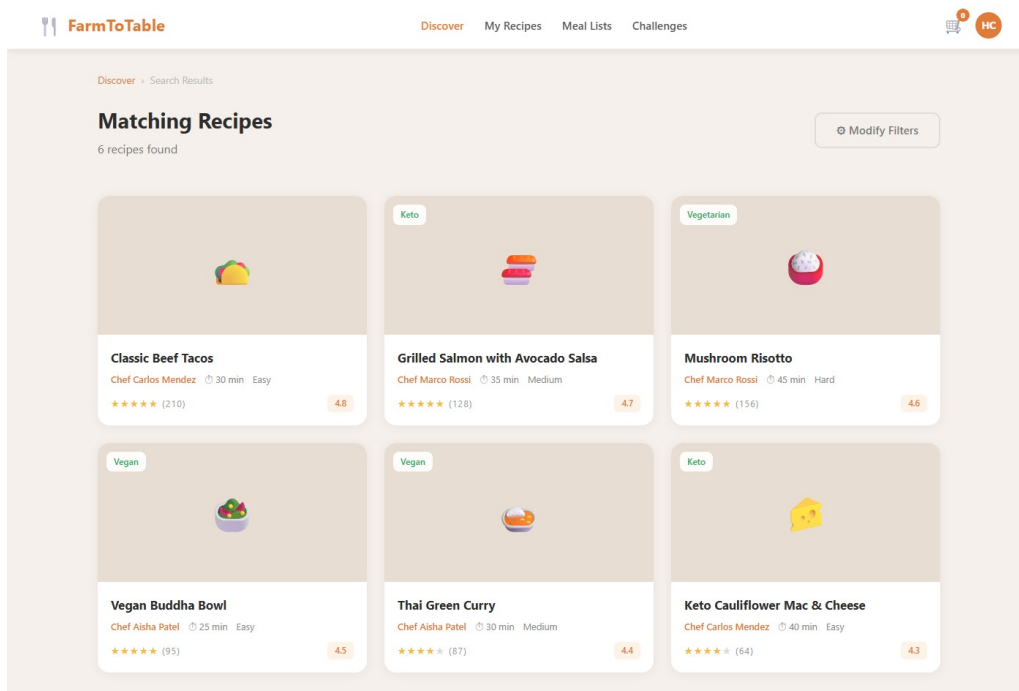
Minimum Chef Rating
 Any Rating

Difficulty
☐ Easy ☐ Medium ☐ Hard

Sort By
 Highest Rated

Clear All Search Recipes

1) Discover Recipes Page



2) Matching Recipes Page

[Discover](#)
[My Recipes](#)
[Meal Lists](#)
[Challenges](#)

[Discover](#)
[Results](#)
Grilled Salmon with Avocado Salsa

Grilled Salmon with Avocado Salsa

Chef Marco Rossi
 4.8 chef rating

35 minutes
 Medium
 (128 reviews)

Keto

A vibrant, healthy dish featuring perfectly grilled Atlantic salmon topped with a fresh avocado and lime salsa. This keto-friendly recipe is packed with omega-3s and healthy fats.

Servings:
-
2 people
+

Shop This Meal

Ingredients (scaled for 2 people)

Atlantic Salmon Fillet	400 g
Avocado	1 pcs
Roma Tomato	2 pcs
Red Onion	0.5 pcs
Lime	1 pcs
Olive Oil	30 ml
Garlic Cloves	3 pcs
Fresh Cilantro	15 g

Instructions

- Pat the salmon fillets dry and season generously with salt, pepper, and a squeeze of lime juice.
- Heat olive oil in a grill pan over medium-high heat until shimmering.
- Place salmon skin-side down and cook for 4-5 minutes until the skin is crispy.
- Flip and cook for another 3-4 minutes until the internal temperature reaches 145°F.
- While the salmon cooks, dice avocado, tomato, and red onion into small cubes.
- Combine diced vegetables with minced garlic, chopped cilantro, lime juice, and a pinch of salt.
- Plate the salmon and spoon the avocado salsa generously on top. Serve immediately.

User Reviews (3)

Sarah M.

★★★★★

Absolutely divine! The salsa is the perfect complement. My family loved it.

2025-12-15

David K.

★★★★☆

Great recipe, but I added a bit more lime. The salmon was cooked perfectly.

2025-11-20

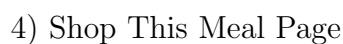
Emma R.

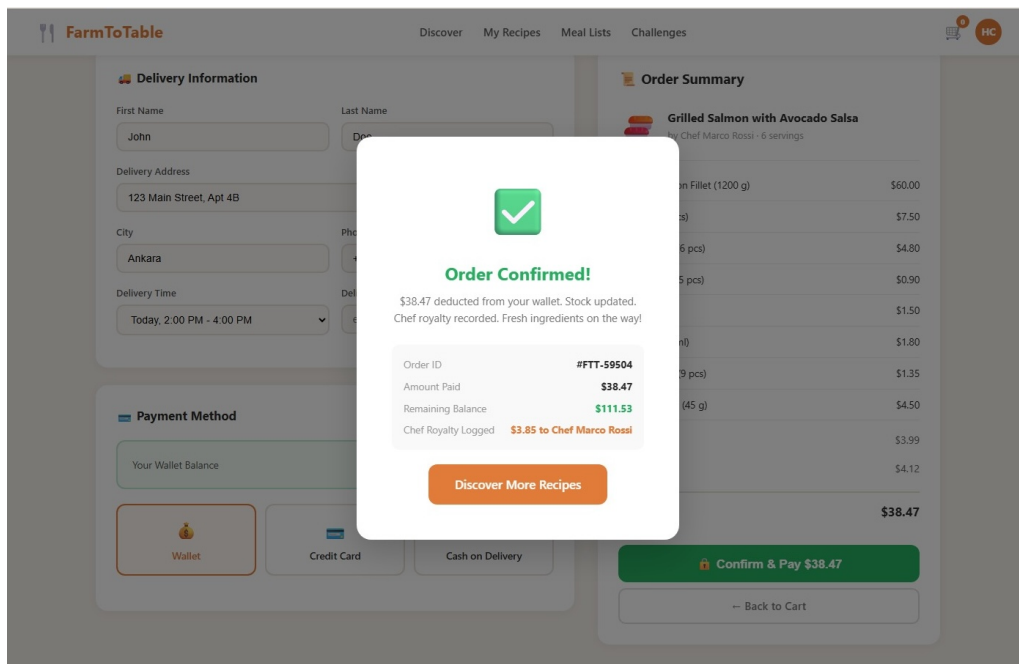
★★★★★

Made this for a dinner party and got so many compliments. Will definitely make again!

2025-10-08

3) Specific Recipe Page





6) Order Confirmed Page

3.3.2 SQL Query: Apply filters based on dietary restrictions, cooking time, ingredients, and chef rating; and list matching recipes

```
-- Search for a Vegan recipe, under 45 minutes, containing "
    Tomato", with a chef rating >= 4.0
SELECT
    r.recipe_id,
    r.recipe_name,
    u.Username AS ChefName,
    r.cooking_time,
    AVG(r.avg-rate) AS average_rating
FROM Recipe r
JOIN CreatorUser cu ON r.creator_id = cu.UserID
JOIN Contains c ON r.recipe_id = c.recipe_id
WHERE r.IsVegan = TRUE
    AND r.cooking_time <= 45
    AND c.ingredient_name = 'Tomato'
GROUP BY
    r.recipe_id,
    r.recipe_name,
    u.Username,
    r.cooking_time
HAVING AVG(r.avg-rate) >= 4.0;
```

3.3.3 SQL Query: The Home Cook selects a recipe to view details

```
--1.c.i
SELECT ingredient_name, quantity, unit
FROM Contains
```

```

WHERE recipe_id = 101;

SELECT u.Username, rt.given_rank, rt.review_text
FROM Rates rt
JOIN User u ON rt.UserID = u.UserID
WHERE rt.recipe_id = 101;

--1.c.ii-iii
SELECT
    c.ingredient_name,
    (c.unit-quantity / r.servings) * 6 AS scaled_quantity, --if
    selected for 6 people
    c.unit
FROM Contains c
JOIN Recipe r ON c.recipe_id = r.recipe_id
WHERE r.recipe_id = 101;

```

3.3.4 SQL Query: User clicks "Shop This Meal"

```

--1.d.i
WITH RecipeIngredients AS (
    SELECT
        c.ingredient_name,
        (c.quantity / r.servings) * 4 AS required_qty -- 4 here
        is for wanted servings by customer
    FROM Contains c
    JOIN recipe r ON c.recipe_id = r.recipe_id
    WHERE r.recipe_id = 101
)
SELECT
    ri.ingredient_name,
    ri.required_qty,
    ls.BusinessName AS SupplierName,
    p.quantity AS available_stock,
    p.price AS unit_price
FROM RecipeIngredients ri
JOIN Provides p ON ri.ingredient_name = p.ingredient_name
JOIN LocalSupplier ls ON p.UserID = ls.UserID
WHERE p.quantity >= ri.required_qty;

--1.d.ii
-- I assumed that missing 'Roma Tomato' ingredient is known and
-- 2.5 unit is required as it says "for an exact ingredient"
SELECT
    ic_sub.ingredient_name AS suggested_substitute_name,
    ic_sub.Ingredient_id AS suggested_substitute_id,
    ls.BusinessName AS SupplierName,
    p.quantity AS available_stock,
    p.price AS unit_price
FROM Ingredient i_orig

```



```

JOIN Ingredient i_sub
    ON i_orig.ingredient-id = i_sub.ingredient-id
    AND i_orig.ingredient_name != i_sub.ingredient_name
JOIN Provides p
    ON i_sub.ingredient_name = p.ingredient_name
    AND i_sub.ingredient-id = p.ingredient-id
JOIN LocalSupplier ls
    ON p.UserID = ls.UserID
WHERE ic_orig.ingredient_name = 'Roma Tomato' -- Name of the
    missing ingredient
    AND ic_orig.ingredient-id = 999 -- ID of missing
    ingredient
    AND p.quantity >= 2.5; -- Quantity of
    needed missing ingredient

--1.d.iii

INSERT INTO MealKit (mealKit_ID) VALUES (10002);

INSERT INTO constructs (mealKit_ID, recipe_id)
VALUES (10002, 101);

INSERT INTO card_contains (HomeCookID, mealKit_ID, supplierID,
    coefficient)
WITH RecipeReqs AS (
    SELECT ingredient_name, (quantity * 4) AS required_qty -- 4
        is selected as coefficient
    FROM Contains
    WHERE recipe_id = 101
),
EligibleSuppliers AS (
    SELECT
        p.UserID AS supplierID,
        SUM(p.price * r.required_qty) AS total_cost,
        COUNT(p.ingredient_name) AS matched_ingredients
    FROM provides p
    JOIN RecipeReqs r ON p.ingredient_name = r.ingredient_name
    WHERE p.quantity >= r.required_qty
    GROUP BY p.UserID
),
CheapestSupplier AS (
    SELECT supplierID
    FROM EligibleSuppliers
    WHERE matched_ingredients = (SELECT COUNT(*) FROM RecipeReqs)
    ORDER BY total_cost ASC
    LIMIT 1
)
SELECT
    501 AS HomeCookID,

```

```

10002 AS mealKit_ID,
supplierID,
4 AS coefficient -- 4 is selected as coefficient
FROM CheapestSupplier;

```

3.3.5 SQL Query: Checkout and Payment

```

BEGIN TRANSACTION;

-- i.

UPDATE User
SET balance = balance - (
    SELECT total_price
    FROM ShoppingCart
    WHERE UserID = 501
)
WHERE UserID = 501
AND balance >= (
    SELECT total_price
    FROM ShoppingCart
    WHERE UserID = 501
);

-- ii.
UPDATE provides p
JOIN (
    SELECT
        cc.supplierID,
        c.ingredient_name,
        SUM(c.quantity * cc.coefficient) AS total_deducted_amount
    FROM card_contains cc
    JOIN constructs co ON cc.mealKit_ID = co.mealKit_ID
    JOIN Contains c ON co.recipe_id = c.recipe_id
    WHERE cc.UserID = 501
    GROUP BY cc.supplierID, c.ingredient_name
) AS cart_items
ON p.UserID = cart_items.supplierID
AND p.ingredient_name = cart_items.ingredient_name
SET p.quantity = p.quantity - cart_items.total_deducted_amount;

-- iii.
INSERT INTO past_purchase (UserID, ShoppingCart_id, date,
    total_fee)
VALUES (
    501,
    1001,
    CURRENT_DATE,

```

```

        (SELECT total_price FROM ShoppingCart JOIN cart_has ON
            ShoppingCart.ShoppingCard_id = cart_has.ShoppingCard_id
            WHERE UserID = 501)
    );

UPDATE VerifiedChef vc
JOIN (
    SELECT
        cr.UserID AS chef_id,
        COUNT(cc.mealKit_ID) AS cooked_count
    FROM card_contains cc
    JOIN constructs co ON cc.mealKit_ID = co.mealKit_ID
    JOIN creates cr ON co.recipe_id = cr.recipe_id
    WHERE cc.UserID = 501
    GROUP BY cr.UserID
) AS cart_chefs ON vc.UserID = cart_chefs.chef_id
SET vc.avgRoyalty = vc.avgRoyalty + (0.50 * cart_chefs.
    cooked_count); -- 0.50 is assumed to be award for one meal

-- v. Empty the user's shopping cart
DELETE FROM ShoppingCart WHERE UserID = 501;

COMMIT;

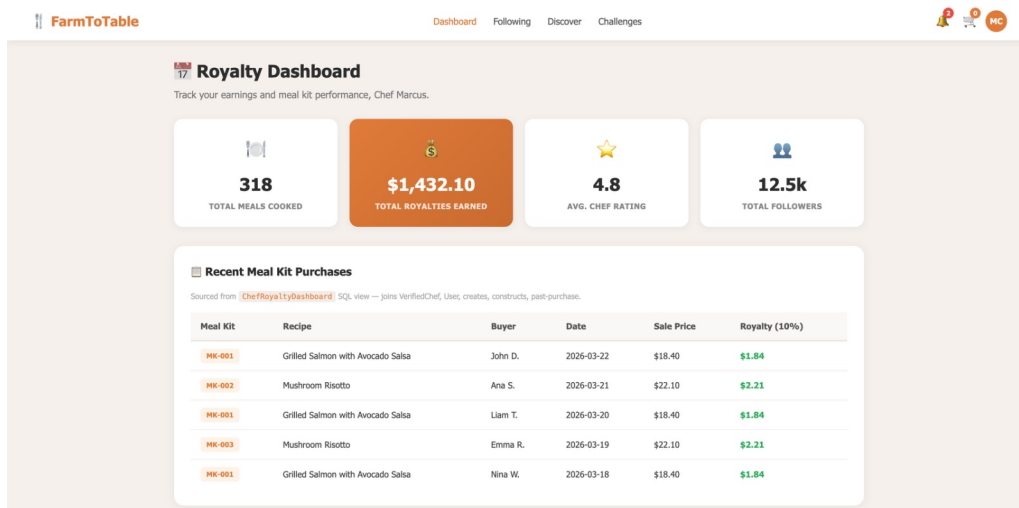
```

4 Advanced Database Components

4.1 Views

4.1.1 Chef Royalty Dashboard View

This view is utilized on the Verified Chef's Royalty Dashboard interface. It aggregates data across five different tables to calculate how many times a chef's recipes have been purchased and displays their current accumulated royalties. Using a view here simplifies the backend code significantly, preventing the need to write this complex JOIN every time the chef refreshes their profile page.

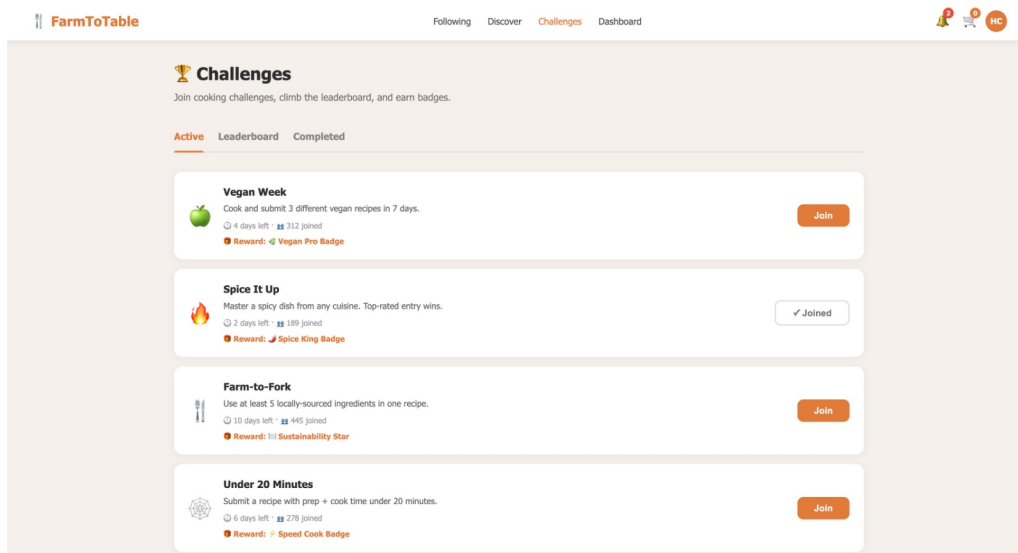


1) Royalty Page

```
CREATE VIEW ChefRoyaltyDashboard AS
SELECT
    vc.UserID AS ChefID,
    u.Username AS ChefName,
    COUNT(pp.mealKit_ID) AS TotalMealsCooked,
    vc.avgRoyalty AS TotalRoyaltiesEarned
FROM VerifiedChef vc
JOIN User u ON vc.UserID = u.UserID
JOIN creates c ON vc.UserID = c.UserID
JOIN constructs co ON c.recipe_id = co.recipe_id
LEFT JOIN 'past-purchase' pp ON co.mealKit_ID = pp.mealKit_ID
GROUP BY vc.UserID, u.Username, vc.avgRoyalty;
```

4.1.2 Active Cart Summary View

This view directly supports the Shopping Cart user interface. When a Home Cook clicks "Shop This Meal" and goes to their cart to confirm payment, the UI needs to display a detailed receipt: which recipe they are buying, the specific ingredients, the scaled quantities, which local supplier is providing them, and the subtotal for each item.



2) Challenges Page

```
CREATE VIEW ActiveCartSummary AS
SELECT
    cc.HomeCookID,
    mk.mealKit_ID,
    r.recipe_name,
    c.ingredient_name,
    (c.quantity * cc.coefficient) AS required_quantity,
    ls.BusinessName AS supplier_name,
    p.price AS unit_price,
    (c.quantity * cc.coefficient * p.price) AS item_subtotal
FROM card_contains cc
JOIN MealKit mk ON cc.mealKit_ID = mk.mealKit_ID
JOIN constructs co ON mk.mealKit_ID = co.mealKit_ID
JOIN recipe r ON co.recipe_id = r.recipe_id
JOIN Contains c ON r.recipe_id = c.recipe_id
JOIN LocalSupplier ls ON cc.supplierID = ls.UserID
JOIN provides p ON cc.supplierID = p.UserID AND c.ingredient_name
    = p.ingredient_name;
```

4.2 Triggers

4.2.1 Automotic Royalty Update

This trigger automatically executes whenever a Home Cook successfully completes a checkout (inserting a row into the past-purchase table). It finds the Verified Chef who created the recipe associated with the purchased MealKit and adds a fixed royalty to their account. We use this trigger to enforce business rules at the database level, ensuring chefs always get paid securely, even if the application backend experiences an error immediately after logging the purchase.

```
CREATE TRIGGER TRG_Update_Chef_Royalty
AFTER INSERT ON 'past-purchase'
```

```

FOR EACH ROW
BEGIN
    UPDATE VerifiedChef
    SET avgRoyalty = avgRoyalty + 0.50
    WHERE UserID = (
        SELECT cr.UserID
        FROM creates cr
        JOIN constructs co ON cr.recipe_id = co.recipe_id
        WHERE co.mealKit_ID = NEW.mealKit_ID
    );
END;

```

4.3 Constraints

4.3.1 Recipe Rating Range

This constraint is used when a Home Cook leaves a review and rates a recipe. Even if the frontend has a dropdown or a star-rating component limited to 5 stars, a malicious user could intercept the API request and submit a rating of "1000", which would ruin the AverageRating calculations across the whole platform. This constraint acts as a strict backend safeguard, ensuring data integrity for the recipe ranking algorithm.

```

ALTER TABLE rates
ADD CONSTRAINT chk_valid_rating CHECK ('given-rank' BETWEEN 1 AND
5);

```

5 Implementation Plan

5.1 Next.js with TypeScript for Backend

- The backend of the application is built using Next.js with TypeScript. As a full-stack framework, Next.js allows API routes to be defined alongside the frontend code, eliminating the need for a separate backend server. TypeScript adds static type checking, which helps prevent runtime errors and ensures consistency across data models and server-side logic. All database queries, business logic, and server-side operations are handled through these API routes.

5.2 Next.js and Tailwind CSS for Frontend

- The frontend is developed using Next.js paired with Tailwind CSS. Next.js provides efficient React-based rendering with built-in support for routing and performance optimizations. Tailwind CSS allows for rapid UI development without the overhead of maintaining separate stylesheet files. This combination ensures a responsive, consistent, and maintainable user interface across different screen sizes.

5.3 PostgreSQL for Database Management

- PostgreSQL serves as the primary database management system, hosted locally during development and deployable to any standard cloud provider in production.

It stores all relational data, including user profiles, recipe inventories, meal kit configurations, and purchase records. PostgreSQL’s support for advanced features such as triggers, views, stored procedures, and complex constraints makes it well-suited for the requirements of this project. Database administration and inspection are carried out through tools such as pgAdmin or the command-line `psql` interface.

5.4 Authentication with NextAuth.js

- User authentication is handled by NextAuth.js, a dedicated authentication library for Next.js applications. It supports credential-based login using email and password, with passwords securely hashed via `bcrypt` before being stored in the PostgreSQL database. Session management is handled through JSON Web Tokens (JWT), and all authentication logic runs on the server side within Next.js API routes, ensuring that sensitive credentials are never exposed to the client.

5.5 Integration and Raw SQL Connections

- The application communicates with the PostgreSQL database through the `pg` (node-postgres) client library. All data operations are routed through the Next.js API layer, which establishes a connection pool to the database and executes raw SQL queries directly. This approach maintains a clear separation between the client and the database while ensuring full compliance with the course requirement of writing and demonstrating raw SQL for all data retrieval and manipulation operations.